MINI PROJECT REPORT

Project 2: Student Record Search System Using Trees

Submitted By: Deepanshu Mittal
Roll Number: 2556446
Course: BTech CSE A
Subject: Data Structures and Algorithms

---

1. Introduction

The Student Record Search System is a menu-driven application developed using the C programming language. It uses a Binary Search Tree (BST) to store and manage student records.

Each student record contains an enrolment ID, student name, and department. The system allows users to insert new records, search for students, delete records, and display student information using different tree traversal techniques.

2. Objectives

The main objectives of this project are:

- To understand the working of a Binary Search Tree.
- To insert and search student records efficiently.
- To delete student records from a BST.
- To display records using inorder, preorder, and postorder traversal.
- To reject duplicate enrolment IDs.
- To understand how insertion order affects BST performance.

3. Problem Statement

Managing student records requires an organized way to store, search, and update information. Searching records one by one can be time-consuming when the number of students increases.

This project uses a Binary Search Tree to organize student records according to their enrolment IDs. It provides basic record management operations through a menu-driven interface.

4. Software and Hardware Requirements

Software Requirements

- Operating System: Windows/Linux
- Programming Language: C
- Compiler: GCC, Code::Blocks, or another C compiler
- Editor: Visual Studio Code or any text editor

Hardware Requirements

- Computer or laptop
- Basic memory and storage
- Keyboard and display

5. Data Structure Used

Binary Search Tree (BST)

A Binary Search Tree is a hierarchical data structure in which each node has at most two children.

- The left subtree contains IDs smaller than the current node's ID.
- The right subtree contains IDs greater than the current node's ID.
- Duplicate enrolment IDs are not allowed in this project.

Student Node Structure

Each node stores:

- Enrolment ID
- Student name
- Department
- Pointer to the left child
- Pointer to the right child

```
typedef struct Student {
    int id;
    char name[50];
    char department[50];
    struct Student *left;
    struct Student *right;
} Student;
```

```c
Student *insert(Student *root, int id, const char name[],
                const char department[]) {
    if (root == NULL) {
        printf("Student inserted successfully.\n");
        return createStudent(id, name, department);
    }

    comparisons++;

    if (id < root->id) {
        root->left = insert(root->left, id, name, department);
    } else if (id > root->id) {
        root->right = insert(root->right, id, name, department);
    } else {
        printf("Duplicate enrolment ID not allowed!\n");
    }

    return root;
}

```

6. Algorithms Used

6.1 Insertion

1. Read the student's enrolment ID, name, and department.
2. Compare the ID with the current node.
3. If the ID is smaller, move to the left subtree.
4. If the ID is greater, move to the right subtree.
5. Insert the new node at the appropriate empty position.
6. If the ID already exists, reject the duplicate record.

6.2 Searching

1. Enter the enrolment ID to search.
2. Compare it with the current node's ID.
3. If both IDs match, display the student record.
4. If the entered ID is smaller, search the left subtree.
5. Otherwise, search the right subtree.
6. If no matching node exists, display a record-not-found message.

6.3 Deletion

The program supports three deletion cases:

- **Leaf node:** Delete the node directly.
- **One child:** Replace the node with its child.
- **Two children:** Replace the node's data with its inorder successor and delete the successor node.

6.4 Tree Traversals

- **Inorder:** Left → Root → Right
- **Preorder:** Root → Left → Right
- **Postorder:** Left → Right → Root

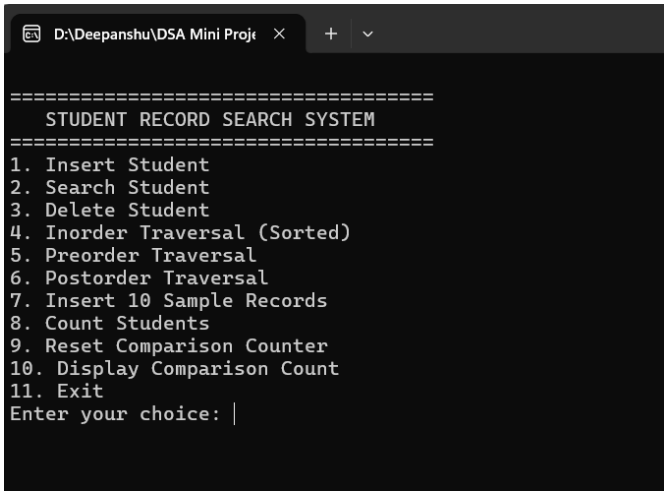
Inorder traversal displays student records in ascending order of enrolment ID.

7. System Implementation

The program is implemented in C using a menu-driven interface. The main operations include:

1. Insert student record
2. Search student record
3. Delete student record
4. Display inorder traversal
5. Display preorder traversal
6. Display postorder traversal
7. Insert sample student records
8. Count student records
9. Reset comparison counter
10. Display comparison count
11. Exit

The program also includes ten sample student records for testing.

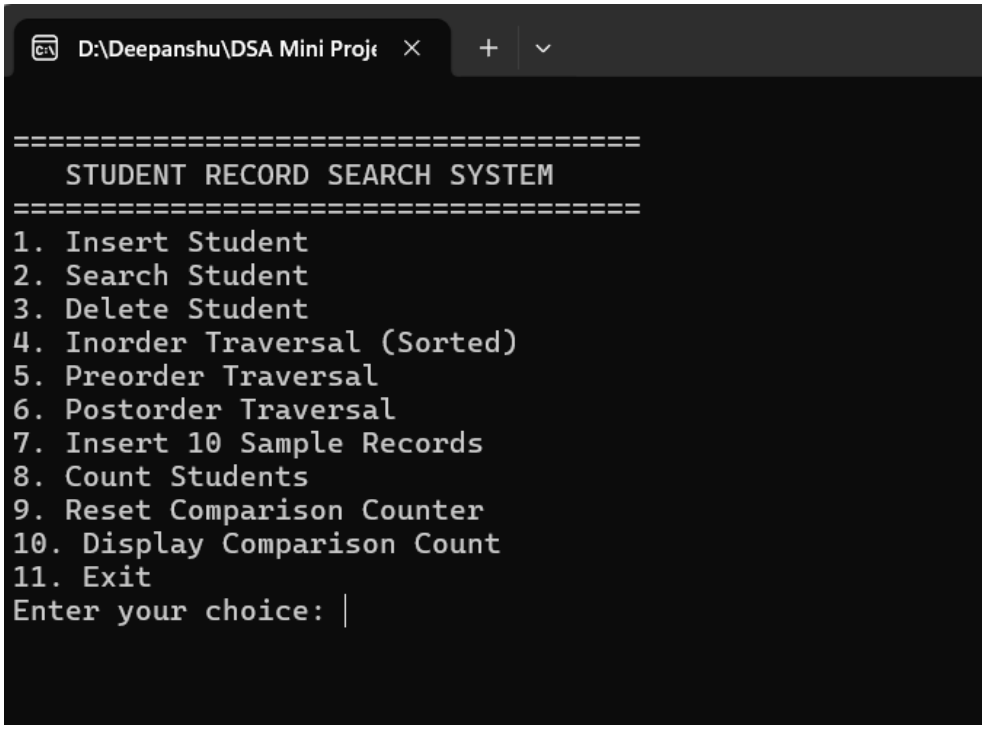


8. Testing and Test Cases

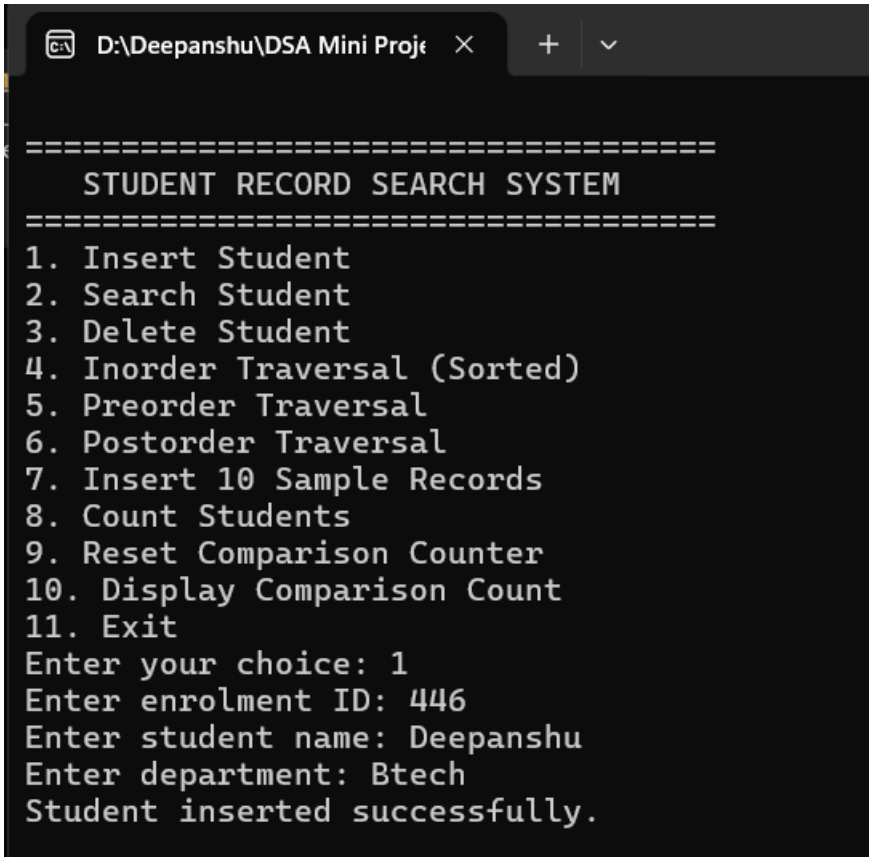
Test Case	Operation	Expected Result
1	Insert a new student	Record is inserted successfully
2	Insert an existing ID	Duplicate ID is rejected
3	Search for an existing ID	Student details are displayed
4	Search for a missing ID	Record-not-found message appears
5	Delete a leaf node	Record is deleted
6	Delete a node with one child	Node is removed and its child is connected
7	Delete a node with two children	Node is removed while BST order is maintained
8	Display inorder traversal	Records appear in ascending ID order
9	Display preorder traversal	Records appear in root-left-right order
10	Display postorder traversal	Records appear in left-right-root order

9. Output Screenshots

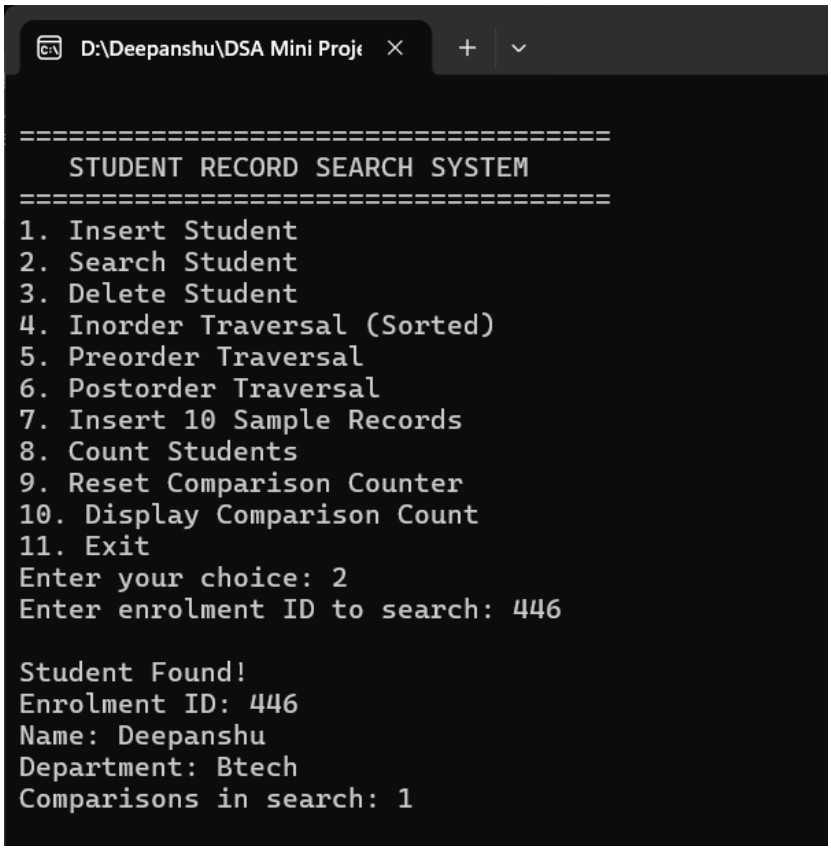
Screenshot 1: Main Menu



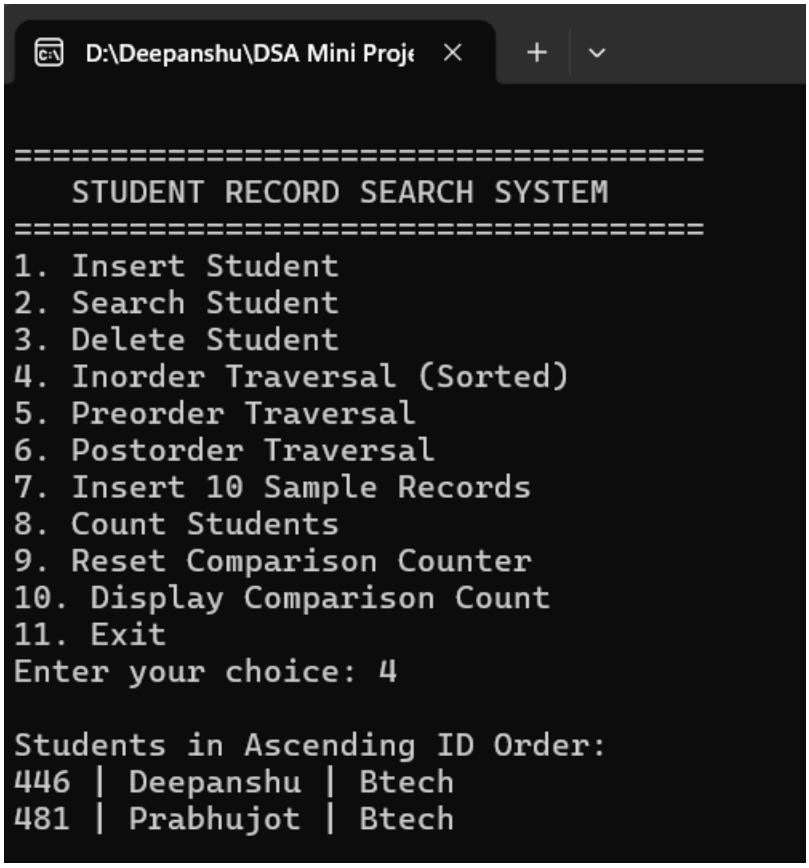
Screenshot 2: Inserting Sample Student Records



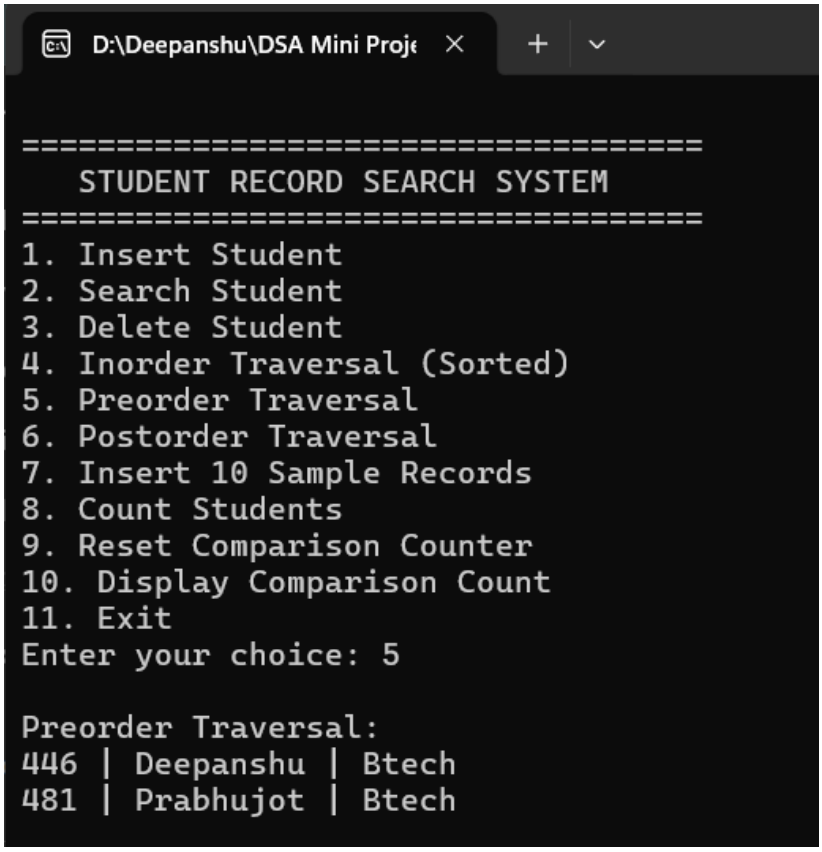
Screenshot 3: Searching for a Student Record



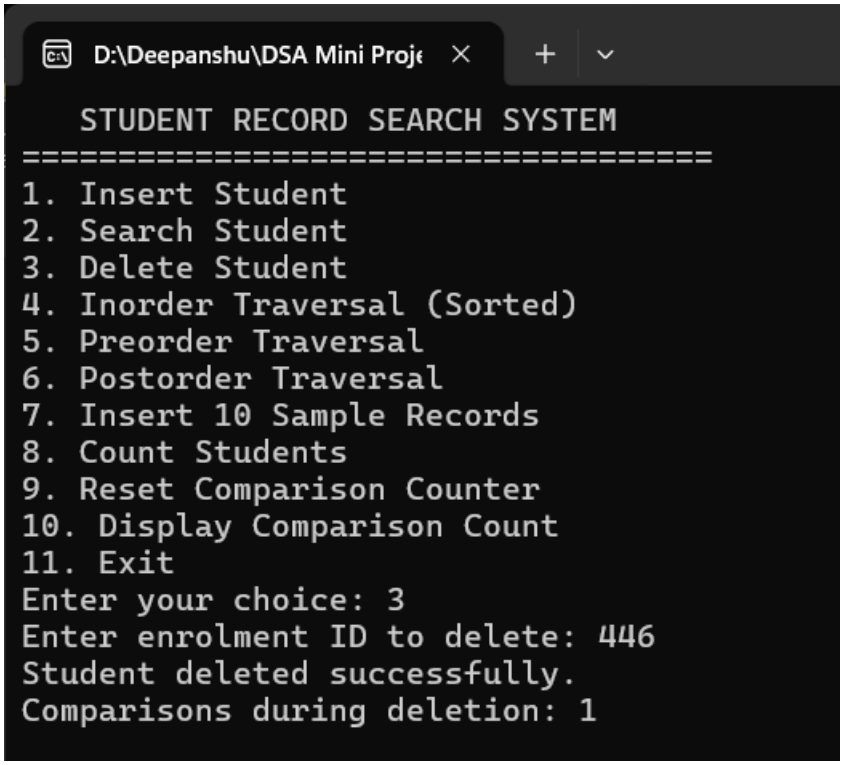
Screenshot 4: Inorder Traversal



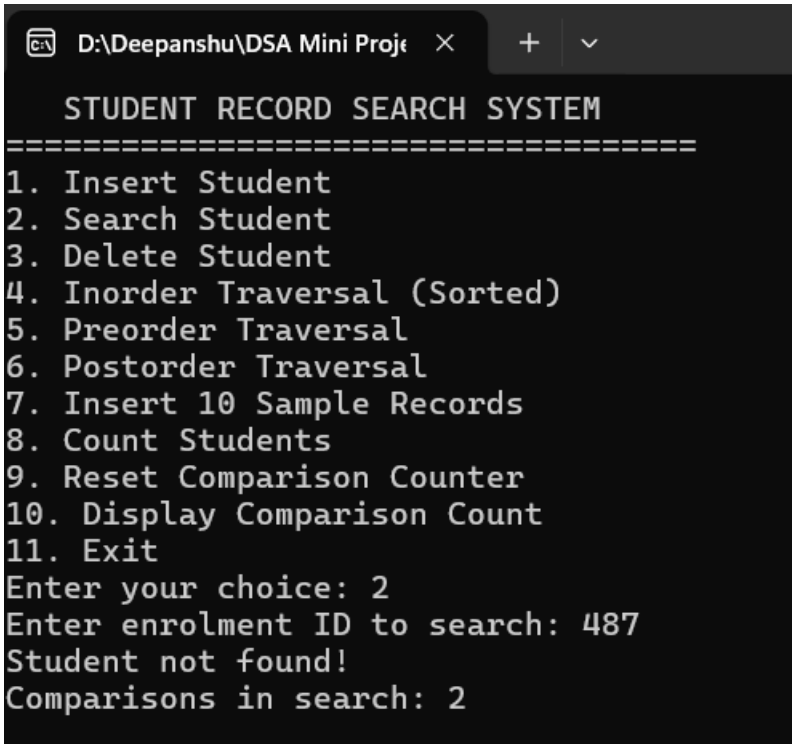
Screenshot 5: Preorder and Postorder Traversals



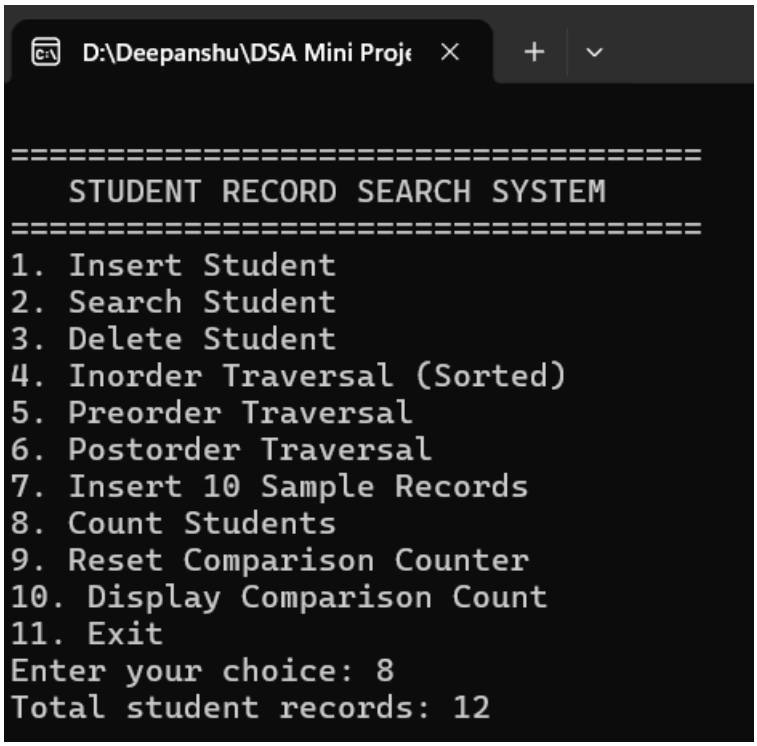
Screenshot 6: Deleting a Student Record



Screenshot 7: Duplicate ID or Record Not Found



Screenshot 8: Comparison Count



10. Time and Space Complexity

Let **n** be the number of student records in the BST.

Operation	Average Case	Worst Case
Insertion	O(log n)	O(n)
Searching	O(log n)	O(n)
Deletion	O(log n)	O(n)
Traversal	O(n)	O(n)

The average-case performance assumes a reasonably balanced tree. In the worst case, the tree becomes skewed, such as when IDs are inserted in sorted order.

The space complexity of storing the BST is **O(n)**. Recursive operations may also require stack space depending on the height of the tree.

11. Comparison of Insertion Orders

The performance of a BST depends on the order in which records are inserted.

Balanced-Looking Insertion Order

When IDs are inserted in an order that keeps the tree reasonably balanced, its height is relatively small. Searching and insertion generally require fewer comparisons.

Sorted Insertion Order

When IDs are inserted in ascending or descending order, the BST may become skewed. Searching can then require up to O(n) comparisons.

The current program counts comparisons during search operations. These counts can be used to observe how different tree shapes affect search performance.

Note: For a fair comparison, create two separate trees containing the same IDs—one inserted in a balanced-looking order and the other in sorted order—and record the comparisons for the same search IDs.

12. BST vs AVL Tree

A Binary Search Tree does not automatically maintain its balance. Its performance depends on the order of insertion and deletion.

An AVL tree is a self-balancing Binary Search Tree. It performs rotations when necessary to maintain balance.

- **BST:** Simpler to implement, but may become skewed.
- **AVL Tree:** More complex because it maintains balance, but provides O(log n) worst-case search, insertion, and deletion.

This project implements a standard BST, not an AVL tree.

13. Advantages

- Simple and menu-driven interface.
- Supports insertion, searching, and deletion.
- Rejects duplicate enrolment IDs.
- Provides three tree traversal methods.
- Demonstrates the effect of tree shape on searching.

14. Limitations

- The tree is not automatically balanced.
- Records are stored in memory and are not saved permanently to a file.
- Performance may decrease if the BST becomes skewed.
- The program supports basic student record operations only.

15. Future Scope

- Add file handling to save records permanently.
- Implement an AVL tree for automatic balancing.

- Add sorting and filtering by department.
- Provide an improved user interface.
- Support updating existing student details.

16. Conclusion

The Student Record Search System demonstrates the practical use of a Binary Search Tree in student record management. It implements insertion, searching, deletion, and tree traversals using the C programming language.

The project also demonstrates that BST performance depends on insertion order. A reasonably balanced tree generally provides better performance than a skewed tree. This project helped in understanding tree-based data structures and their applications.

17. References

1. Data Structures and Algorithms course notes.
2. C programming language reference materials.
3. Binary Search Tree concepts and traversal algorithms.
4. GCC compiler documentation.